ÉCHECS ET DATA SCIENCE

Guide du code, de la construction du jeu de données et de l'analyse statistique

Projet Python • pandas • NumPy • SciPy • statsmodels • Lichess • SSP Cloud

Problématique : dans quelle mesure le choix et la répétition des ouvertures influencent-ils les performances des joueurs en ligne, indépendamment de leur niveau Elo, de leur couleur et de la cadence ?

1. Architecture et principe de non-destruction

Les fichiers existants sont conservés. Les modules de collecte enrichie, nettoyage, statistiques et stockage sont ajoutés. Les cellules anciennes du notebook Hot Hand restent présentes ; sept cellules complémentaires documentent l'étude des ouvertures. Un pipeline indépendant permet de relancer l'étude sans modifier l'ancien protocole.

config_projet.py

Charge .env, initialise les répertoires, lit les cinq graines et la destination S3.

Creation_openings_complet.py

Télécharge les TSV A à E du dépôt Lichess, associe code ECO, nom, variante et famille, puis exporte openings_complet.parquet.

Collecte_enrichie.py

Découvre des joueurs depuis les cinq racines puis extrait les parties au format NDJSON, avec les caractéristiques des deux adversaires.

Nettoyage_donnees.py

Déduplique, normalise les dates, calcule Elo relatif, expérience antérieure, répétition et sessions.

Analyse_ouvertures.py

Calcule descriptifs, tests chi-deux, correction FDR, modèles logistiques, comparaison d'ajustement et graphiques.

Stockage_SSPCloud.py

Téléverse et récupère les Parquet / résultats dans un espace MinIO compatible S3 sans effacer les objets historiques.

Pipeline_ouvertures.py

Orchestre collecte, nettoyage, analyse, et upload optionnel.

2. Création de la base : définitions et précautions

L'unité d'observation principale est le couple (user_id, game_id). Une même partie peut apparaître avec deux points de vue si les deux joueurs ont été collectés. Les doublons exacts de ce couple sont supprimés ; les parties distinctes ne sont pas artificiellement fusionnées. La variable result prend Win, Loss ou Draw ; score vaut respectivement 1, 0 ou 0,5.

Elo relatif : elo_diff = rating - opponent_rating. Le score attendu associé est 1/(1 + 10^(-elo_diff/400)). Cette métrique résume une information cruciale : gagner avec 200 points Elo de plus n'a pas le même sens que gagner avec 200 points Elo de moins.

La famille d'ouverture est issue prioritairement de la classification renvoyée par Lichess. Le dictionnaire externe fournit une taxonomie descriptive cohérente. Les transpositions entre lignes rendent la classification à partir des seuls premiers coups particulièrement fragile ; c'est pourquoi aucune classification forcée n'est appliquée lorsqu'une partie n'a pas d'étiquette.

Les colonnes experience_opening_prior et prev_result_same_color sont construites après tri chronologique. Elles utilisent exclusivement les parties antérieures du même utilisateur, de la même couleur, et, pour l'expérience, de la même famille. Cela évite une fuite temporelle. La répétition à couleur identique constitue une comparaison plus sensée que la répétition brute entre coups de Blancs et de Noirs.

3. Représentativité et limites de l'échantillonnage

La découverte par parcours du graphe d'adversaires autour de cinq graines ne constitue PAS un sondage aléatoire simple : les adversaires de joueurs forts ou actifs auront une probabilité d'inclusion spécifique. Il faut inspecter les distributions Elo dans chaque branche, le nombre de joueurs, la cadence et les dates. Inclure plusieurs graines améliore potentiellement la diversité, mais n'en garantit pas l'équilibre.

PLAN D’ANALYSE STATISTIQUE

4. Statistiques descriptives

Le code calcule taux de victoire, taux de nulle et score moyen par ouverture, couleur, cadence, bande Elo et graine. Les distributions d’Elo et les taux de valeurs manquantes sont des diagnostics préalables. Les familles rares sont regroupées dans Autres pour éviter un grand nombre de groupes presque vides.

5. Tests du chi-deux d’indépendance

H0 : les variables des lignes et colonnes sont statistiquement indépendantes. On construit la table de contingence avec pandas.crosstab ; SciPy calcule chi2_contingency, la statistique et les degrés de liberté. Pour les cellules d’effectif observé O_ij et attendu E_ij = n_i. n_.j / n, la statistique est somme (O_ij-E_ij)^2/E_ij. Elle est confrontée à une loi du chi-deux à (r-1)(c-1) degrés de liberté.

Les tests portent sur résultat × ouverture, couleur, cadence, bande Elo, origine de collecte ; ils sont aussi répétés dans plusieurs sous-groupes. On étudie séparément résultat précédent × répétition, ainsi que répétition × résultat actuel. Le V de Cramér vaut racine de chi2/(n × min(r-1,c-1)). Il quantifie l’importance de l’association. Les petites fréquences attendues fragilisent l’approximation asymptotique ; valid et expected_lt5 signalent ce risque.

La procédure Benjamini-Hochberg corrige les p-values pour comparaisons multiples. Une p-value, même très faible, n’est ni une taille d’effet ni une preuve de causalité. Les observations provenant de joueurs répétés et parfois des mêmes parties peuvent compromettre les hypothèses d’indépendance du chi-deux : ces tests constituent donc un premier outil de description, et non l’inférence finale.

6. Régressions logistiques : construction progressive

Réponse Y = 1 si victoire, 0 si perte ou nulle. On modélise logit[P(Y=1|X)] = beta0 + somme(beta_j X_j). L’odds ratio est exp(beta_j). Tous les modèles M0 à M5 sont estimés sur un échantillon commun et utilisent des erreurs-types regroupées par joueur.

Modèle	Contrôles ajoutés
M0	Couleur seulement
M1	+ différence Elo du duel
M2	+ cadence
M3	+ famille de l’ouverture
M4	+ logarithme de l’expérience passée
M5	+ interaction expérience × couleur

AIC, BIC, pseudo-R2 et test de rapport de vraisemblance comparent les modèles imbriqués. Une baisse d’AIC après ajout de la famille d’ouverture signale une meilleure qualité d’ajustement pénalisée pour le nombre de paramètres, pas une contribution causale démontrée. Une interaction positive expérience × couleur dépend de la catégorie de référence.

7. Répétition, comportement et succès

Régression R : probabilité de réutiliser la même famille à couleur identique, selon la victoire ou la nulle précédente, puis ajout de l’Elo, de la couleur, de la cadence et de l’expérience. La comparaison principale est la défaite précédente, toutes choses égales par ailleurs.

Régression P : probabilité de victoire à la partie actuelle selon répétition de la famille, en ajoutant successivement couleur, Elo, cadence, famille et expérience. Une association positive de répétition avec victoire est compatible avec un effet de familiarité, mais également avec la préférence persistante pour les ouvertures des meilleurs joueurs.

8. Tests de robustesse et prolongements

L'analyse distingue la victoire stricte du score moyen, qui valorise les nulles à 0,5. Une régression sur parties décisives permet de tester la robustesse du résultat. Le code exporte la surprise de performance par famille, score observé moins score attendu Elo. Pour améliorer une future version : effet fixe joueur, analyse par cohorte et période, splits temporels, erreurs à double regroupement partie/joueur et étude de transpositions sur échiquier avec python-chess.

REPRODUCTION ET STOCKAGE SSP CLOUD

9. Installation et exécution

1. Installer les bibliothèques avec python3 -m pip install -r requirements.txt.

2. Copier .env.example vers .env et remplacer les quatre placeholders par les pseudos réellement retenus.

3. Créer le référentiel avec python3 -m Scripts_generation_data.Creation_openings_complet.

4. Lancer python3 Pipeline_ouvertures.py --users-per-seed 30 --games-per-format 60 --upload.

5. Ouvrir Analyse_data.ipynb. Les sept nouvelles cellules figurent après les 65 anciennes.

6. Pour réanalyser les Parquet déjà collectés, utiliser --skip-collect.

10. Pourquoi SSP Cloud ?

MinIO/S3 est un stockage objet persistant, utile quand le notebook local ou la session de calcul est temporaire, pour partager les jeux de données et reproduire les analyses. Chaque fichier est adressé par bucket et clé. Exemple : s3://tristant/diffusion/ouvertures_v2/Data/games_clean.parquet. Le nom du bucket reprend le projet historique, sans garantir que les droits actuels permettent d'y écrire.

Les bibliothèques boto3 et s3fs savent communiquer avec un endpoint compatible S3. Dans un service SSP Cloud les identifiants sont souvent injectés dans les variables d'environnement. Hors SSP Cloud, il faut renseigner endpoint, clé d'accès et secret dans .env, jamais dans un script ni dans Git. Le script charge automatiquement les identifiants présents dans l'environnement, n'active pas de politique publique et n'efface pas la sauvegarde précédente.

11. Lire les résultats correctement

Après exécution, ouvrir Resultats/tests_chi2.csv : examiner p_fdr_bh, cramer_v et expected_lt5. Dans les fichiers modeles_*_coefficients.csv, examiner OR et IC95%, avec attention à la catégorie de référence. Dans les comparaisons, tester le gain d'ajustement entre M2 et M3 pour la contribution descriptive des ouvertures, et l'évolution du coefficient de répétition entre P0 et P4.

Aucun effet statistique chiffré n'est avancé ici : la collecte réelle n'a pas été exécutée dans le présent environnement. Le code fournit les calculs et les sorties dès qu'une base valide lui est fournie. Ne pas réutiliser les conclusions quantitatives du notebook Hot Hand comme des résultats de cette nouvelle problématique.

12. Sécurité et précision

Le fichier d'inspection initial comportait un jeton Lichess en clair. La copie fournie utilise désormais une variable d'environnement. Le jeton ancien doit être révoqué depuis le compte Lichess et remplacé. Aucune authentification SSP Cloud, API Lichess ni téléversement effectif n'a été vérifié pendant cette préparation.

Références

API Lichess : https://lichess-org.github.io/api/

Ouvertures officielles Lichess : https://github.com/lichess-org/chess-openings

Documentation stockage SSP Cloud : https://docs.sspcloud.fr/content/storage.html